
GNN Over-Squashing and Virtual Nodes

Mario Kroll Merino
ICAI
202113781@alu.comillas.edu

Abstract

Graph Neural Networks (GNNs) relying on standard message passing struggle to capture long-range dependencies due to the over-squashing phenomenon, where information from a growing receptive field is compressed into fixed-capacity node representations. This project investigates the efficacy of global graph rewiring to mitigate over-squashing in bottlenecked topologies. Graph Convolutional Networks (GCN) and Graph Isomorphism Networks (GIN) are evaluated against Virtual Node-augmented counterparts using the synthetic Tree-NeighborsMatch dataset. Results demonstrate that standard architectures suffer a performance collapse at a tree depth of 4. Introducing a Virtual Node delays this collapse and yields a 5x relative improvement in test accuracy for the GIN architecture at depth 4. All evaluated models fail at depth 5. It concludes that a Virtual Node reduces the graph’s structural diameter to 1 but replaces the structural bottleneck with an aggregation bottleneck. This overwhelms the global node’s fixed-size representation capacity when faced with a dense neighborhood. The code is available at <https://github.com/mariokroll/gnn-oversquashing-analysis>.

1 Introduction

Graph Neural Networks (GNNs) are the standard architecture for learning on relational data. Most GNNs utilize a message-passing paradigm where nodes update their representations by aggregating information from their local neighborhoods. While effective for localized tasks, this paradigm struggles when target predictions depend on long-range interactions.

This limitation can be driven by the over-squashing phenomenon. As the number of layers increases to expand a node’s receptive field, the volume of information grows exponentially, particularly in tree-like topologies. This information must be compressed into a fixed-size vector representation. Consequently, signals from distant nodes become distorted or lost, degrading model performance.

Graph rewiring techniques are commonly proposed to alleviate this bottleneck. This project investigates one specific intervention: the addition of a Virtual Node [1]. A Virtual Node connects to all other nodes in the graph, reducing the structural diameter to one. This creates a direct mechanism for global information exchange and bypasses topological bottlenecks.

In this work, the efficacy of the Virtual Node intervention is evaluated. To do so, the synthetic Tree-NeighborsMatch dataset [2] is utilized to isolate a model’s ability to propagate long-range signals. Then, Graph Convolutional Networks (GCN) and Graph Isomorphism Networks (GIN) are benchmarked against their Virtual Node counterparts. The empirical findings indicate that while the Virtual Node delays performance collapse by mitigating structural bottlenecks, it ultimately suffers from an aggregation bottleneck at higher tree depths.

2 Related Work

Alon and Yahav (2021) [2] formalized the over-squashing phenomenon, demonstrating that standard architectures suffer performance collapse on long-range tasks. This degradation occurs because information from an exponentially growing receptive field must be compressed into fixed-capacity node representations, causing distant signals to be diluted. Topping et al. (2022) [3] provided a geometric understanding of this issue. They linked over-squashing to local graph topology using discrete Ricci curvature, proving that negatively curved edges act as severe structural bottlenecks.

To mitigate this issue, graph rewiring methods can offer a solution. Graph rewiring methods modify the input topology to improve information flow. There has been numerous approaches to this, including adding edges based on node similarity, random rewiring and spectral methods. Attali et al. (2026) [4] comprehensively surveyed many techniques, noting a persistent trade-off: methods that increase edge density to solve over-squashing frequently exacerbate over-smoothing and introduce high computational overhead.

The introduction of a Virtual Node serves as a specialized, global rewiring intervention. By connecting a central node to all existing entities, the graph’s structural diameter is uniformly reduced to one. Recent variations include Local Virtual Nodes (2025) [5], which attach hubs only to high-centrality regions to preserve the global topological structure. Our work builds upon this subfield by empirically testing the capacity limits of a global Virtual Node under extreme density conditions.

3 Background and Preliminaries

3.1 Graph Notation

Let $G = (V, E)$ be an undirected graph, where V is the set of nodes and E is the set of edges. Each node $v \in V$ has an associated feature vector $\mathbf{x}_v \in \mathbb{R}^d$. The neighborhood of a node v is defined as $\mathcal{N}(v) = \{u \in V : (u, v) \in E\}$.

3.2 Message Passing Neural Networks

Message Passing Neural Networks (MPNNs) are a class of GNNs that operate by iteratively updating node representations through a message-passing mechanism. At each layer, nodes aggregate messages from their neighbors and update their hidden states accordingly. The general update rule for a node v at layer l can be expressed as:

$$\mathbf{h}_v^{(l)} = \text{UPDATE}^{(l)} \left(\mathbf{h}_v^{(l-1)}, \text{AGGREGATE}^{(l)} \left(\{\mathbf{h}_u^{(l-1)} : u \in \mathcal{N}(v)\} \right) \right) \quad (1)$$

where $\mathbf{h}_v^{(l)}$ is the hidden representation of node v at layer l , and $\text{AGGREGATE}^{(l)}$ and $\text{UPDATE}^{(l)}$ are learnable functions that depend on the specific GNN architecture. The initial state $\mathbf{h}_v^{(0)}$ is typically set to the node’s feature vector $\mathbf{x}_v$. Two established architectures are evaluated in this work:

- **Graph Convolutional Networks (GCN):** GCNs utilize a simple linear transformation followed by a non-linear activation function to update node representations. The aggregation is typically performed using a normalized sum of neighboring node features.
- **Graph Isomorphism Networks (GIN):** GINs are designed to be as powerful as the Weisfeiler-Lehman graph isomorphism test. They use a multi-layer perceptron (MLP) for the update function and include a learnable parameter to control the contribution of the central node’s own features.

3.3 The Tree-NeighborsMatch Dataset

The Tree-NeighborsMatch dataset is a synthetic benchmark designed to evaluate a model’s ability to capture long-range dependencies. It consists of binary trees where each node has a tuple consisting of key and value. Every leaf node is assigned a random key and a hidden class value. The root node is assigned a target key but no value. The objective is node classification on the root node. To predict the correct class, the model must locate the leaf node containing the matching key and propagate its associated value to the root. This forces a strict long-range dependency of length k . Figure 1 shows

an example of this dataset. Because the number of leaves grows exponentially with k , the topology forces a structural bottleneck where multiple signals merge during propagation toward the root. This dataset is ideal for isolating the over-squashing phenomenon, as it creates a controlled environment where the only way to succeed is to effectively propagate information across a large receptive field.

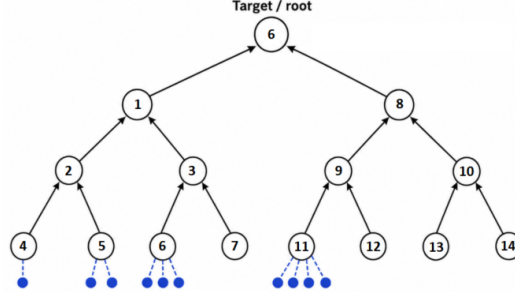


Figure 1: Example of a Tree-NeighborsMatch instance with $k = 3$. The root node (top) must predict the class of the leaf node (bottom) that shares the same key. The model must propagate information from the correct leaf through the internal nodes to the root, demonstrating the need for long-range information flow.

4 Methodology

4.1 Model Architectures

Two base architectures are evaluated: Graph Convolutional Networks (GCN) and Graph Isomorphism Networks (GIN). Node input features in the Tree-NeighborsMatch task consist of discrete key and value identifiers. Given the bounded vocabulary size required for a maximum tree depth of 5, these features are encoded using one-hot vectors. For an input graph of N nodes, this yields an input feature matrix $X \in \{0, 1\}^{N \times d}$. While using an embedding layer is common for discrete data, the use of one-hot vectors passed directly into the first convolutional layer is mathematically equivalent and computationally negligible for this scale of synthetic task. Both architectures utilize multiple graph convolutional layers, standardizing on a hidden dimension of 64 throughout the network.

4.2 Virtual Node Intervention

To mitigate over-squashing bottlenecks, a virtual node intervention is introduced. This is implemented through a global state update paradigm rather than sparse edge modification to prioritize computational efficiency during batched training. This paradigm establishes a global information routing mechanism while preserving the optimized sparse matrix operations inherent in standard GNN implementations. Figure 2 illustrates the architecture of the virtual node intervention.

4.2.1 Initialization

At the beginning of each forward pass, a virtual node $vn^{(0)}$ is initialized to a tensor of all zeros. This represents a blank slate for global information aggregation. This decision ensures that the virtual node does not carry any prior bias and allows it to reflect the structural context of the current tree. The capacity to learn is confined to the weights of the associated neural networks rather than the initial state vector itself.

4.2.2 Global State Update

After the initial message-passing layer, the virtual node executes, for each remaining layer l the following steps:

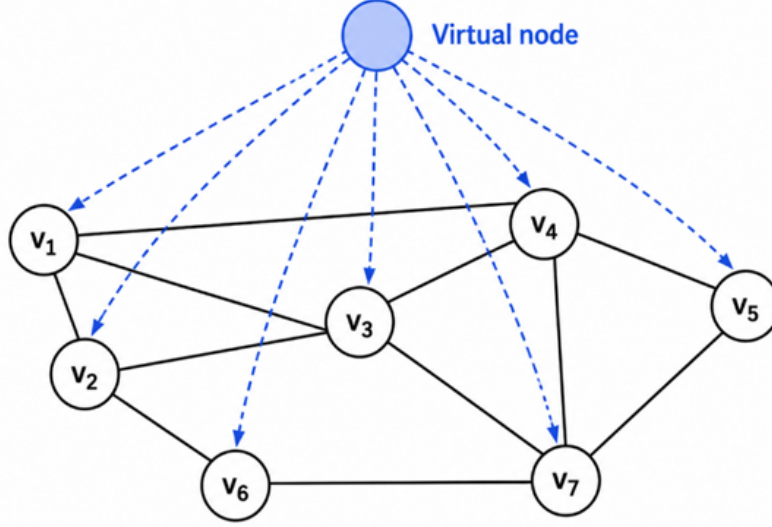


Figure 2: Architecture of the Virtual Node intervention. The virtual node (VN) aggregates information from all nodes in the graph and broadcasts an updated global state back to all nodes at each layer after the first.

1. **Aggregation:** The virtual node aggregates information from all other nodes in the graph. This is implemented as a global mean pooling operation for this aggregation:

$$p^{(l)} = \frac{1}{|V|} \sum_{v \in V} \mathbf{h}_v^{(l)} \quad (2)$$

2. **Update:** The virtual node’s representation is updated using a 2-layer MLP with ReLU activation and Batch Normalization. The MLP takes the aggregated representation as input and produces an updated global state:

$$vn^{(l)} = \text{MLP}^{(l)}(vn^{(l-1)} + p^{(l)}) \quad (3)$$

3. **Broadcast:** The updated virtual node representation is then broadcast back to all other nodes in the graph. This is implemented as a simple addition operation to each node in V :

$$\mathbf{h}_v^{(l)} = \mathbf{h}_v^{(l-1)} + vn^{(l)} \quad (4)$$

4.3 Implementation Choices

4.3.1 Pooling/Broadcasting vs Edge Insertion

When thinking about how to implement the virtual node, two main approaches were considered: pooling/broadcasting and edge insertion. The pooling/broadcasting approach was chosen for its computational efficiency. By using global mean pooling to aggregate information and a simple addition operation for broadcasting, the implementation avoids the overhead of modifying the graph structure and can leverage optimized sparse matrix operations. In contrast, edge insertion would require dynamically modifying the graph’s adjacency matrix, which can be computationally expensive and less efficient during training. This approach achieves the same functional outcome of enabling global information exchange while maintaining the efficiency of standard GNN implementations.

4.3.2 Asymmetric Design

The virtual node operations begin after the first message-passing layer, creating an asymmetric design. The first GNN layer works as an input encoder, transforming the discrete one-hot features into a continuous hidden representation. A symmetric design would establish communication earlier, but this approach allows the virtual node to operate on a more informative representation of the graph, as well as reduce model parameters and computational overhead.

5 Results

To compare the effectiveness of the virtual node intervention, both GCN and GIN architectures were evaluated on the Tree-NeighborsMatch dataset across varying tree depths. Both models were trained vanilla and with the virtual node intervention. They were all trained using the same hyperparameters, so that results are directly comparable.

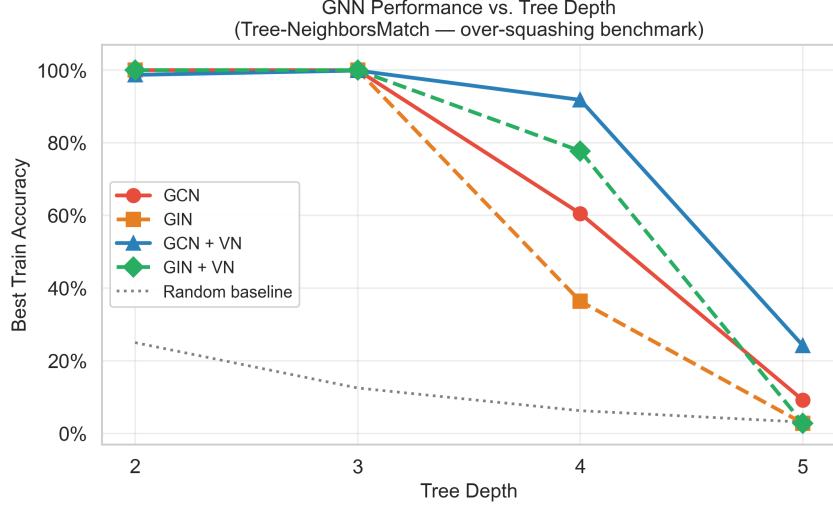


Figure 3: Accuracy of the models on the training set for each tree depth.

To measure the impact of the virtual node, the accuracy of the models on the training and test sets was recorded for each tree depth. To evaluate the over-squashing phenomenon, bearing in mind the computational demands of training trees with higher depths, since it grows exponentially, the depths considered were [2,3,4,5]. The results indicate that both GCN and GIN architectures suffer a performance collapse at a tree depth of 4 when trained without the virtual node. This is consistent with the over-squashing phenomenon, as the models struggle to propagate information from the exponentially growing receptive field. On the other hand, the introduction of the virtual node significantly delays this collapse. Figure 3 shows the accuracy of the 4 models on the training set for each of the depths studied in this experiment. Instead of using the test set for evaluating this models, since the number of trees generated increases with the tree depth, the performance of the models in the test set with depth 2 was poor and it overfitted the training data. Since it was a replication of the experimental conditions from Alon et al. (2021) [2], it was decided to use the training set for comparison. For the GIN + VN architecture, the train accuracy at depth 4 improves from less than 40% to almost 80%, representing a 2x relative improvement. As for the GCN + VN architecture the accuracy also improves, though to a lesser extent. However, the GCN + VN architecture shows a more robust behavior, outperforming the rest of the models in all cases. Even at depth 5, although the results are poor, GCN + VN performs much better than the other models, who struggle (or fail) to beat the random abaseline. All evaluated models fail at depth 5, indicating that while the virtual node mitigates structural bottlenecks, it ultimately suffers from an aggregation bottleneck when faced with a dense neighborhood.

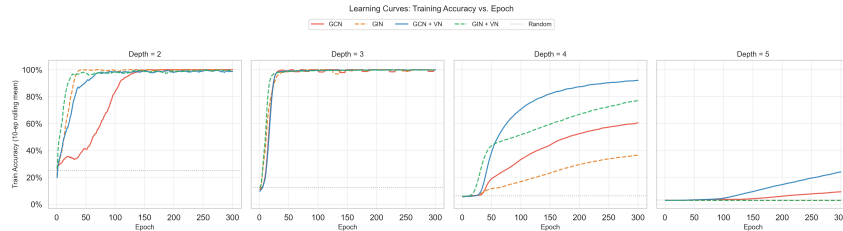


Figure 4: Learning curves of the models on the training set for each tree depth.

As for the convergence of the models, all of them converged within 100 epochs, with the GIN + VN architecture showing a faster convergence compared to the other models. This suggests that the virtual node not only improves the final performance but also facilitates the learning process by providing a more direct path for information flow. The 4 shows the training accuracy of the models across epochs for each tree depth.

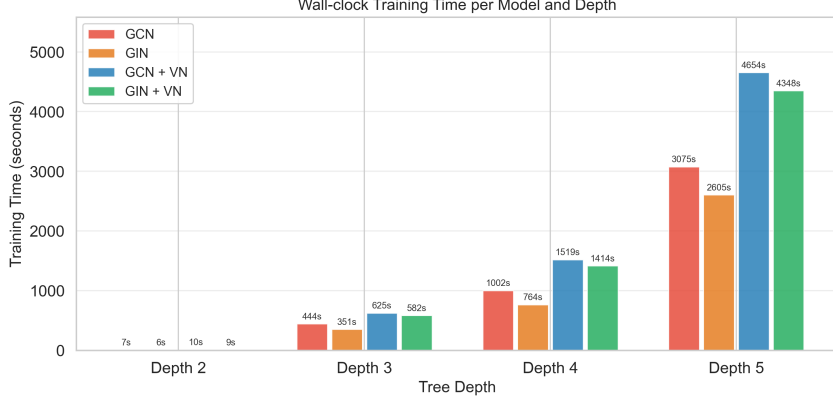


Figure 5: Training time of the models on the training set for each tree depth.

Regarding the training time, the introduction of the virtual node does increase the computational overhead due to the additional operations for aggregation and broadcasting. The larger the tree depths, the more pronounced the effect becomes. This happens because the virtual node introduces additional computations at each layer, which accumulate as the tree depth increases. For depth 4, the increase in training time compensates for the improved performance. As for depth 5, this is no longer the case, because all 4 models have a bad performance, and the increased training time does not justify the marginal performance gain. The Figure 5 shows the total training time for each model across tree depths.

6 Discussion

The results shown in Section 5 validate the core hypothesis at intermediate tree depths. The virtual node intervention significantly mitigates the over-squashing phenomenon, allowing models to maintain high accuracy at depth 4 where standard architectures collapse. This demonstrates that the virtual node effectively reduces structural bottlenecks by providing a direct path for global information exchange. However, the failure of all models at depth 5 indicates that while the virtual node addresses structural bottlenecks, it may introduce an aggregation bottleneck. As the virtual node aggregates information from all other nodes, its fixed-size representation can become overwhelmed when faced with a dense neighborhood, leading to a new bottleneck. This suggests that while the virtual node is a powerful intervention for mitigating over-squashing, it may not be sufficient for tasks requiring extremely long-range dependencies. In general terms, the results highlight the trade-offs involved in graph rewiring interventions. While they can alleviate structural bottlenecks, they may introduce new challenges such as increased computational overhead and potential aggregation bottlenecks. The choice of intervention should be guided by the specific requirements of the task and the underlying graph topology.

On the other hand, the GCN + VN architecture shows a more robust behavior across all tree depths, outperforming the other models even at depth 5. This suggests that the GCN architecture may be better suited to leverage the global information provided by the virtual node, potentially due to its simpler aggregation mechanism compared to GIN. This observation warrants further investigation into how different GNN architectures interact with global rewiring interventions and whether certain designs are more effective at mitigating over-squashing without introducing new bottlenecks. In this project, we chose GCN and GIN as representative architectures of standard GNNs. More than one was necessary to prove that the observed effects were not specific to a single architecture, but it was found unnecessary to evaluate more architectures, as the results were consistent across both models. Furthermore, the introduction of more complex architectures, together with adding a virtual node

to each one of them would have made the experiment more computationally too expensive, without providing significant additional insights.

Future work could explore more sophisticated global aggregation mechanisms or hybrid approaches that combine local and global rewiring strategies to further alleviate bottlenecks in GNNs. There has been already some work in this direction, such as the Local Virtual Nodes [5], which attach hubs only to high-centrality regions to preserve the global topological structure while still providing a mechanism for long-range information flow. One might also consider implementing a hierarchical virtual node structure, where multiple virtual nodes are introduced at different levels of the graph to capture information at varying scales. This could potentially alleviate the aggregation bottleneck by distributing the global information across multiple nodes rather than relying on a single virtual node. However, this would also introduce additional complexity and computational overhead, so careful design and experimentation would be necessary to evaluate its effectiveness.

7 Conclusion

This work investigates the efficacy of Virtual Nodes in mitigating over-squashing in Graph Neural Networks using the synthetic Tree-NeighborsMatch dataset. The empirical findings demonstrate that while Virtual Nodes effectively reduce structural bottlenecks by enabling direct global information exchange, they introduce new challenges at extreme graph densities. Both GCN and GIN architectures with Virtual Nodes significantly outperform their vanilla counterparts at tree depth 4, with GIN+VN achieving a 2x relative improvement in accuracy. However, all evaluated models fail at depth 5, revealing an aggregation bottleneck that limits the capacity of the global node’s fixed-size representation. The GCN+VN architecture shows greater robustness across all depths, suggesting that simpler aggregation mechanisms may better exploit global information. These results highlight the fundamental trade-offs in graph rewiring interventions: while they alleviate structural limitations, they may introduce computational overhead and new forms of information compression bottlenecks. Future research should explore hybrid approaches combining local and global rewiring strategies, or hierarchical virtual node structures, to achieve more effective long-range dependency propagation in GNNs.

8 AI Usage

This project was developed using a combination of human expertise and AI assistance. All Gemini, Claude, ChatGPT and Github Copilot were used to assist in various stages of the project, including literature review (Gemini), methodology design (Gemini + Claude), code implementation and error debugging (Claude Code + Codex), and report supervising and correction (Gemini + ChatGPT). The AI tools were used as assistants to enhance the efficiency and quality of the work, but all final decisions, interpretations, and conclusions were made by the human author. The code implementation was primarily done by the human author, with AI assistance for specific coding tasks and debugging. The report writing was also done by the human author, with AI assistance for structuring and refining the text. Overall, the project represents a collaborative effort between human expertise and AI capabilities.

9 References

- [1] Pham, T., Tran, T., Dam, H., & Venkatesh, S. (2017). Graph Classification via Deep Learning with Virtual Nodes. ArXiv.org. <https://arxiv.org/abs/1708.04357>
- [2] Alon, U., & Yahav, E. (2021, March 9). On the Bottleneck of Graph Neural Networks and its Practical Implications. ArXiv.org. <https://doi.org/10.48550/arXiv.2006.05205>
- [3] Topping, J., Di Giovanni, F., Chamberlain, B. P., Dong, X., & Bronstein, M. M. (2022, November 12). Understanding over-squashing and bottlenecks on graphs via curvature. ArXiv.org. <https://doi.org/10.48550/arXiv.2111.14522>
- [4] Graph Rewiring in GNNs to Mitigate Over-Squashing and Over-Smoothing: A Survey. (2026). Arxiv.org. <https://arxiv.org/html/2411.17429v2#bib.bib100>
- [5] Local Virtual Nodes for Alleviating Over-Squashing in Graph Neural Networks. (2025). Arxiv.org. <https://arxiv.org/html/2508.20597v1>